



Интеграция облачной АТС для очередей и агентов контакт-центра

Delivered by: ASRP · **Role:** ASRP owned the PBX-integration workstream — design, implementation, and hardening · **Engagement type:** Independent contractor · **Domain:** Cloud telephony for freight logistics

Исполнитель: ASRP · **Роль:** ASRP полностью вела направление интеграции с АТС — проектирование, реализацию и доведение до боевой устойчивости · **Тип engagement:** независимый подрядчик · **Домен:** облачная телефония для грузовой логистики

1. Краткое резюме

ASRP спроектировала и построила интеграцию, позволяющую современной платформе коммуникаций работать поверх **внешней облачной АТС** — RingLogix / Netsapiens¹. Организации, чья телефония уже живёт на этой АТС, продолжают пользоваться своими существующими **очередями АТС (huntgroups), агентами и устройствами — настольными телефонами, мобильными и софтбоками**, при этом поставленная платформа становится единой **системой учёта для пользовательского интерфейса**: агенты входят и выходят из очередей, выбирают устройство, видят, кто на линии, и забирают звонки — всё с одного веб-экрана, без переключения на отдельный агентский портал самой АТС.

Основной инженерный вызов в том, что **истина разделена между двумя системами**. Внешняя АТС является источником истины для **живого** состояния телефонии (кто реально находится в очереди, какое устройство сейчас зарегистрировано); база данных платформы является источником истины для всего, что читает UI. ASRP построила механизм, который удерживает эти два представления в сходящемся состоянии: **управление состоянием очередей и агентов, синхронизацию устройств, корреляцию звонков**, связывающую событие звонка от АТС с нужным звонком платформы, и **самовосстанавливающийся слой сверки** (входящие вебхуки событий плюс плановый «жнец»/reaper), который устраняет расхождение всякий раз, когда живое событие было пропущено.

Это был **самый крупный проект в рамках сотрудничества** — существенная подсистема, которую ASRP провела от пустой схемы до рабочей системы, проверенной на реальной АТС, и фактически **подсистема с единственным владельцем** внутри более крупного командного проекта (см. §13).

2. Бизнес-контекст и проблема

Многие контакт-центры строят свою телефонию на хостируемой облачной АТС и не собираются от неё отказываться — номера, отношения с оператором связи и настольные телефоны агентов живут именно там. Но собственная агентская функциональность АТС скудна, и это отдельный мир от современной веб-платформы, где на самом деле живут диспетчеры, супервайзеры и аналитика.

Цель состояла в том, чтобы позволить этим организациям **сохранить телефонию на АТС, но управлять ею из UI платформы** — без замедления UI из-за походов к АТС при каждом обновлении экрана и без незаметного расхождения двух систем при потере события. Конкретно:

- **Агентам** нужно входить и выходить из очереди АТС и выбирать, какое устройство звонит, прямо из веб-UI.
- **Супервайзерам** нужно живое, достоверное представление о том, кто доступен, а кто на звонке.
- **Входящие звонки**, маршрутизированные АТС в очередь, должны появляться как полноценные звонки на платформе, чтобы агент мог увидеть их и забрать со всем контекстом.
- Платформа должна **оставаться корректной, даже когда АТС пропускает уведомление** — никаких «зависших» логинов-призраков, никакого застрявшего состояния «на звонке».

Сквозное жёсткое требование: экраны должны быть **быстрыми и всегда отвечающими**, а зеркало состояния АТС должно быть **со временем корректным, даже когда сеть ведёт себя не идеально**.

3. Что было поручено ASRP

У платформы уже была модель звонков, веб-UI и собственная встроенная маршрутизация очередей. Мандат ASRP заключался в том, чтобы аккуратно пристроить к этому целый мир *внешней АТС*:

1. **Смоделировать домен АТС** в базе данных платформы — очереди, устройства агентов, состояние входа для каждого агента — в виде специально спроектированных таблиц плюс несколько колонок в существующих записях.
2. **Дать агентам управлять очередями АТС из UI** — вход/выход из очереди с корректным выбором устройства и предоставление софтбокса прямо в браузере.
3. **Сделать звонки из очередей АТС полноценными** на платформе — сопоставлять каждое событие звонка от АТС с нужным звонком платформы и автоматически назначать его отвечающему агенту.
4. **Поддерживать честность зеркала** — слой сверки, который восстанавливает состояние базы данных из событий АТС, и, как резервный механизм, плановая задача, сверяющая состояние с авторитетным представлением АТС.

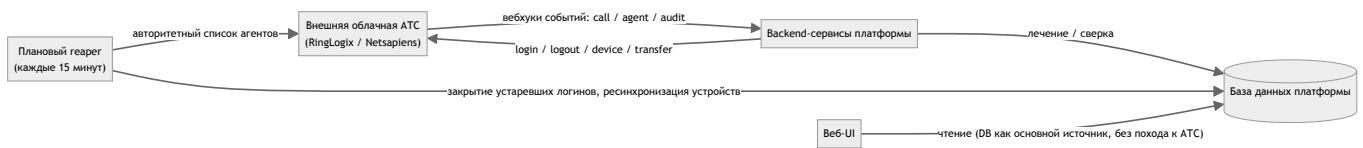
Четыре возможности ниже (§4–§6) — это то, что было поставлено; §7 рассказывает истории о том, что шло не так по пути.

4. Архитектура системы

Дизайн намеренно **асимметричен в вопросе о том, кто владеет истиной**, и именно эта асимметрия — ключевая идея:

- **Внешняя АТС** является источником истины для *живого* внешнего состояния — реального членства в очереди и регистрации устройств.
- **База данных платформы** является источником истины для **чтения в UI**. Экраны читают из базы данных, никогда не обращаясь к АТС на «горячем пути», поэтому UI остаётся быстрым и всегда отвечает.

- **Три пути сходимости** поддерживают точность зеркала в базе данных: **прямые мутации** (когда сама платформа выполняет вход/выход/перевод звонка), **входящие вебхуки событий** (АТС проталкивает события по звонкам/агентам/аудиту, которые почти в реальном времени лечат базу данных) и **плановый reaper** (периодическая задача, сверяющая состояние с авторитетным списком агентов АТС, чтобы уловить всё пропущенное).



Почему зеркало + reaper + replay, а не чтение АТС в реальном времени?

Очевидный наивный дизайн: при каждом рендере экрана спрашивать у АТС, кто залогинен и какие устройства зарегистрированы. ASRP сознательно отвергла этот путь по трём причинам:

- **Задержка и радиус поражения.** Дашборд супервайзера обновляется постоянно. Размещение стороннего REST-вызова на каждом чтении делает UI таким же медленным — и таким же *доступным* — как АТС в её худший день. Оборванный или задресселенный ответ АТС обнулил бы весь экран.
- **Лимиты запросов и стоимость.** API АТС — это общий ресурс с ограничением частоты запросов. Направление к нему каждого чтения UI не масштабируется на множество одновременных супервайзеров.
- **UI нужны данные, которых у АТС нет.** Состояние «забрано», история, контекст по организации и корреляция с записями звонков платформы существуют только на стороне платформы.

Поэтому платформа держит **локальное зеркало**, которое можно читать мгновенно, и рассматривает самую сложную часть — *поддержание истинности зеркала* — как явную многопутевую задачу сходимости, а не игнорирует её. Вебхуки делают зеркало свежим; reaper делает его *со временем корректным* даже при потере вебхука; прямые мутации поддерживают корректность на счастливом пути. Ни один путь не является единой точкой отказа. Вся архитектура уместается в одном предложении: **читать локально, непрерывно сходиться и никогда не позволять шагу сверки разрушить достоверное живое состояние.**

Аутентификация к АТС

Платформа общается с АТС через её REST API по протоколу **OAuth2**, с токеном, кэшируемым в Redis, который проактивно и безопасно обновляется под конкурентной нагрузкой (дедупликация внутри процесса плюс распределённая блокировка), а также с прозрачным одноразовым повтором при истечении срока действия токена. *(Более глубокие технические детали механики обновления/блокировки доступны по NDA.)*

5. Звонок простым языком (пример сценария)

Вымышленный, но достоверный по отношению к реальному потоку. *Дана* — агент поддержки в грузовой брокерской компании, чья телефония работает на внешней АТС.

1. **Дана входит в систему.** Она открывает платформу, выбирает очередь «Dispatch» и переключает свой статус на доступный. Платформа считывает её устройства, автоматически выбирает её настольный телефон (она пользовалась им последний раз), подтверждает у АТС, что настольный телефон действительно состоит в очереди Dispatch, сообщает АТС о её входе, записывает запись о входе и перечитывает состояние АТС, чтобы убедиться, что обе стороны согласны. Её статус мгновенно становится зелёным на экране каждого супервайзера.
2. **Поступает звонок.** Перевозчик набирает номер брокерской компании. Платформа маршрутизирует звонок в очередь Dispatch АТС и — что важно — помечает запись звонка как *звонок из очереди АТС* и записывает единственный идентификатор, о котором договорились и АТС, и медиашлюз (см. §6). Звонок появляется в очереди на экране Даны как доступный для взятия.
3. **Телефон Даны звонит, и она отвечает.** АТС прозванивает очередь, её настольный телефон звонит, она берёт трубку. АТС генерирует событие звонка («этот звонок теперь активен на добавочном номере 2XX»). Платформа сопоставляет это событие со звонком на экране Даны по общему идентификатору, определяет, *какому именно агенту* принадлежит добавочный номер 2XX, помечает звонок как «Соединён — Дана» и фиксирует время ответа.
4. **Что-то теряется.** Предположим, АТС так и не отправила событие «Дана вышла из системы» в конце смены. Через пятнадцать минут reaper обходит активные логины, напрямую спрашивает АТС «Дана всё ещё залогинена в Dispatch?», получает «нет» и закрывает устаревший логин — никакого агента-призрака, который бы продолжал светиться зелёным.
5. **Вариант с софтоном.** Если бы Дана выбрала телефон прямо в браузере вместо настольного телефона, платформа предоставила бы ей выделенное SIP-устройство на АТС — остальной поток идентичен.

6. Корреляция звонков (самая сложная часть)

Самой сложной частью интеграции было удержание в реальном времени синхронности **состояния телефона, состояния очереди и живого представления супервайзера об обоих**: когда АТС маршрутизирует входящий звонок в очередь, этот звонок должен появиться на платформе как реальный, доступный для взятия звонок, а каждое последующее событие АТС о нём должно попасть на **правильный** звонок платформы и корректно дойти до экранов агента и супервайзера — без какой-либо гарантии порядка прихода этих сигналов относительно друг друга.

Механизм, который делает это надёжным, описан в инженерном справочнике ASRP. Более глубокие технические детали доступны по NDA.

7. Инженерные истории из боевых условий

Самыми интересными частями этого проекта были сбои, которые проявляются только против живой сторонней системы. Стоит рассказать о трёх — каждая по схеме проблема → диагностика → исправление.

7.1 Событие, опередившее нашу запись (гонка при корреляции)

Проблема. Звонки из очередей АТС отвечаются *очень* быстро. В продакшене АТС генерировала своё событие звонка, объявляющее звонок в очереди уже активным, **до того**, как платформа успевала дописать ключ корреляции для этого звонка. Поиск из §6 не находил ничего, и событие

звонка — то самое, что назначает звонок отвечающему агенту, — просто терялось. Звонки отображались как незабранные на экране, хотя человек уже разговаривал.

Диагностика. Это классическая распределённая гонка: два независимых производителя (медиашлюз, дающий нам ключ корреляции, и АТС, дающая нам событие для корреляции) без гарантии порядка между ними. На медленных звонках ключ выигрывал благодаря удаче со временем; на быстрых звонках из очереди выигрывало событие. Исправить это простым «пишите быстрее» было невозможно.

Исправление — буферизация и повтор. Когда событие звонка от АТС не может быть сопоставлено немедленно, оно **буферизуется в Redis под идентификатором звонка с TTL около 1 часа** вместо того, чтобы быть отброшенным. В момент, когда ключ корреляции всё же записывается (из вебхука шлюза или из резервного вызова API шлюза, добавленного именно потому, что звонки из очередей отвечаются слишком быстро для вебхука), платформа **повторно проигрывает** буферизованные события для этого звонка. Путь повтора **атомарно опустошает буфер**, повторно обрабатывает каждое событие с **изоляция ошибок по элементам** и **ставит обратно в очередь** любой элемент, который всё ещё не удаётся сопоставить (продлевая TTL), вместо того чтобы его терять. Итоговый эффект: ранние, поздние и дублирующиеся события АТС безопасны, а корреляция становится не зависящей от порядка. Этот путь буферизации и повтора — самая доработанная часть всей интеграции.

7.2 База данных, отставшая от АТС (обратное лечение)

Проблема. Зеркало было построено на неявном предположении, что платформа «опережает» АТС — сначала она логинит агента, затем АТС подтверждает. Но расхождение случается и в *обратном* направлении: устройство агента перерегистрируется, и АТС считает его залогиненным в очередь, а в базе данных платформы **нет вообще никакой записи о входе** (пропущенное событие, перезапуск, ручное изменение на стороне АТС). Агент был активен на АТС, но невидим на платформе.

Диагностика. Считать базу данных строго нижестоящей по отношению к действиям, инициированным платформой, было ошибкой. АТС авторитетна для живого состояния, поэтому *платформа должна быть готова узнавать состояние, которое сама не создавала*.

Исправление — лечение в обоих направлениях. Когда приходит событие регистрации устройства, платформа проверяет авторитетный список агентов АТС; если АТС показывает, что агент залогинен, а в базе данных нет логина, платформа **создаёт недостающую запись о входе** («обратное лечение») и завершает обработку раньше, чтобы общий обработчик не перезаписал её немедленно. Теперь лечение работает симметрично — вебхуки могут как *создавать* недостающий логин, так и *закрывать* недействительный, — и каждое решение проверяется по авторитетному списку АТС, чтобы устаревшее уведомление не могло создать фантомный логин. В той же области было добавлено выделенное правило **защиты активного звонка**: агент, который выглядит «недоступным», но на самом деле находится в середине звонка, никогда не выводится из системы — его состояние разрешается событием звонка.

7.3 Взятие звонка, пережившее неудачный перевод (откат)

Проблема. Взятие звонка из очереди АТС — это действие в два шага: пометить звонок как взятый в базе данных, затем сказать АТС перевести живой звонок на устройство агента. Если второй шаг завершился неудачей — перевод возвращал неуспех, или отсутствовали необходимые данные

маршрутизации, — то *первый* шаг уже был зафиксирован. Результатом становился **тихий ложноположительный результат**: платформа показывала звонок взятым агентом, который на самом деле никогда не был соединён.

Диагностика. Локальная мутация и удалённый побочный эффект трактовались так, будто они всегда завершаются успешно вместе. Это не так, и локальное состояние не должно переживать неудачный удалённый шаг.

Исправление — снять взятие и выбросить ошибку. Путь взятия звонка теперь трактует перевод ATC как точку невозврата. Если `transfer-to-device` возвращает неуспех, **или** выбрасывает исключение, **или** отсутствуют необходимые данные (домен / добавочный номер / идентификатор корреляции), платформа **откатывает локальное взятие** (снимает пометку «взято») и **поднимает ошибку неудачного взятия**, чтобы UI отражал реальность. Всё взятие целиком сериализуется под распределённой блокировкой, так что два агента не могут взять один и тот же звонок одновременно. Ложное состояние «взято» больше не может сохраняться за неудачным переводом.

7.4 Всё ещё дорабатывается (честно названо)

Для организаций, охватывающих **несколько очередей**, существует нюанс разграничения источника истины для чтения/записи, который ASRP определила как область для дальнейшей доработки — он назван здесь, а не замят.

8. Технологический стек

Слой	Технология
Внешняя облачная ATC (цель интеграции)	RingLogix / Netsapiens ² (REST API, OAuth2)
Backend	TypeScript на Node.js / Bun
Доступ к данным	Drizzle ORM (TypeScript, из общего пакета схемы)
Хранение данных	MySQL — PlanetScale serverless в продакшене, драйвер mysql2 против MySQL 9 для локальной разработки, с разделением на primary/replica
Кэширование, блокировки, буферы	Redis — кэш токенов, распределённые блокировки (Redlock), буферы повтора событий корреляции
Реалтайм в UI	NATS pub/sub → WebSocket → фронтенд
Планирование	Cron-сервис (reaper сверки)

«Какую базу данных вы использовали?» → MySQL, через Drizzle ORM в TypeScript. Продакшен работает на PlanetScale serverless (совместимом с MySQL); локальная разработка использует драйвер mysql2 против MySQL 9. Используется разделение primary/replica, и чтения состояния очереди, которые должны быть авторитетными, закреплены за primary.

9. Модель данных

Интеграция опирается на реляционную схему MySQL (Drizzle ORM). ASRP спроектировала следующие специально созданные таблицы (описаны концептуально — проприетарные имена и колонки обобщены):

- **Реестр устройств** — одна строка на устройство АТС на пользователя: состояние регистрации (зарегистрировано / не зарегистрировано), контакт регистрации / IP / user-agent / срок действия, очередь, к которой относится устройство, и метки времени последнего появления / последнего выбора. Уникальность обеспечена по (организация, пользователь, устройство).
- **История регистрации устройств** — журнал снимков регистрации только для добавления, для целей аудита, записываемый только тогда, когда снимок действительно меняется.
- **Текущие записи о входе в очередь** — одна строка на активный логин, с **уникальным ограничением, гарантирующим не более одного активного логина на (организация, пользователь, очередь)**, плюс устройство, статус и метка времени входа.
- **История входов в очередь** — архив закрытых логинов, чтобы таблицы живого состояния оставались маленькими, а история сохранялась.
- **Реестр очередей (huntgroups)** — набор очередей на организацию, связанный с конфигурацией номера телефона и флагом активности.

ASRP также добавила небольшое число **колонок в существующие таблицы платформы**: идентификатор домена АТС на организации, идентификатор добавочного номера АТС на пользователе, флаг «этот номер — очередь АТС» на номере, и — в записи звонка — **индексированный ключ корреляции** и флаг звонка из очереди АТС, используемые для корреляции звонков (§6), плюс поля назначения отвечающего агента и времени ответа.

Доступ к данным централизован в типизированных вспомогательных функциях запросов (создание / закрытие / замена логина, синхронизация устройств, разрешение организации по домену АТС и пользователя по добавочному номеру АТС), с авторитетными чтениями состояния очереди, закреплёнными за основной базой данных.

10. Надёжность и эксплуатация

- **Трёхпутевая сходимость** — прямая мутация, вебхуки событий и плановый gearer вместе поддерживают согласованность зеркала с АТС; ни один путь не является единой точкой отказа.
- **Самовосстановление в обоих направлениях** — вебхуки могут как *создавать* недостающий логин (обратное лечение, §7.2), так и *закрывать* недействительный; gearer независимо сверяется с авторитетным списком агентов АТС каждые **15 минут**, с единственным исполнением под распределённой блокировкой Redis.
- **Защита активного звонка** — сверка никогда не выводит из системы агента, который действительно находится на звонке, даже если временный сигнал предполагает обратное.
- **Инвариант одного активного логина** — обеспечивается на уровне базы данных (уникальное ограничение) и транзакционными вспомогательными функциями входа / замены, так что гонка не может породить два активных логина для одной очереди.

- **Общее определение «эффективного входа»** — единый предикат определяет, что на самом деле значит «залогинен» (с учётом состояний доступен / вручную / автоматически и того, доставляется ли звонок), используемый одинаково UI, вебхуками и gearer, чтобы они не могли разойтись.
- **Конкурентно-безопасная обработка токенов** — обновление OAuth2 защищено промисом внутри процесса и распределённой блокировкой Redis, с проактивным обновлением и прозрачным одноразовым повтором при 401.
- **Устойчивость к событиям не по порядку и дубликатам** — буфер повтора событий корреляции делает ранние, поздние или дублирующиеся события АТС безопасными; неудачные элементы изолируются и ставятся обратно в очередь, а не отбрасываются (§7.1).
- **Устойчивые к сбоям взятия звонков** — неудачный перевод АТС откатывает локальное взятие, вместо того чтобы оставить ложное состояние «взято» (§7.3).
- **Реалтайм-рассылка** — изменения агента / очереди публикуются через NATS для каждой релевантной аудитории UI (для конкретного пользователя и для группы), чтобы экраны супервайзера и агента оставались живыми.

11. Зрелость и статус

Честное разделение того, на каком этапе находится работа.

Построено и итеративно доработано (четыре возможности): вход/выход агента из очереди с автовыбором устройства, синхронизация устройств и предоставление софтбокса, корреляция звонков с автоматическим назначением агента, и слой самовосстановления на вебхуках плюс gearer. Всё это реализовано и было проверено на живой внешней АТС.

Доведено до боевой устойчивости (в рамках отдельного инженерного цикла в середине 2026 года): устойчивость буфера повтора корреляции, распределённая блокировка для gearer и обновления токена, привязка выбора устройства к huntgroup, упрощение до чтений в первую очередь из базы данных на горячем пути и транзакционная модель входа. ASRP также провела самостоятельно инициированный проход по анализу и устранению недостатков подсистемы в рамках этого цикла.

Планы на будущее: нюанс разграничения чтения/записи для нескольких очередей из §7.4 — это область дальнейшей доработки, а не поставленная гарантия; он не блокировал результат, описанный ниже, поскольку применяется только к организациям, охватывающим несколько очередей.

В целом. Интеграция достигла рабочего состояния, проверенного на живой АТС, и прошла пилот у клиента (см. §12 о том, что случилось дальше). Развитие этой конкретной подсистемы сейчас **приостановлено:** не из-за технического препятствия, а из-за решения клиента в пользу собственной разработки после пилота, принятого после пилота (§12). Сама возможность остаётся поддерживаемой частью платформы для организаций, уже работающих на телефонии этой внешней АТС. Эта страница не заявляет о статусе продакшена в масштабе или общей доступности.

12. Результаты и итог

Интеграция была построена и проверена от начала до конца против живой внешней АТС: агенты входили в очереди АТС из веб-UI, входящие звонки из очереди появлялись как доступные для взятия звонки и автоматически назначались отвечающему агенту, а слой самовосстановления устранял расхождение состояния после пропущенных событий.

Проект дошёл до стадии пилота у клиента. На этом этапе экономика клиента склонилась в пользу другого пути: маршрутизация звонков через собственный **встроенный движок очередей** платформы оказалась дешевле в эксплуатации, чем пропуск продакшен-трафика через внешнюю АТС. Поскольку агентский UI уже был построен и проверен от начала до конца на интеграции с внешней АТС, переключение этой организации на внутреннюю разработку потока очередей платформы позволило переиспользовать те же инвестиции во фронтенд, а не начинать с нуля — фронтенд не нужно было перестраивать, только перенаправить на внутренний движок маршрутизации.

Это было **решение клиента в пользу собственной разработки, принятое после пилота**, а не техническая неудача интеграции — возможность внешней АТС работала так, как задумано, и остаётся доступной на платформе для организаций, уже привязанных к работе телефонии на этой внешней АТС. Никакие показатели внедрения или производительности не заявляются за пределами пилота, поскольку ни один не был измерен на продакшен-трафике в масштабе.

13. Что поставила ASRP

ASRP полностью вела **всё направление интеграции с внешней АТС** поверх существующей платформы, и это фактически была **подсистема с единственным владельцем** внутри более крупного командного проекта — история коммитов по этим путям подавляющим большинством принадлежит одному участнику (ASRP) против небольшого числа коммитов от одного другого члена команды, что согласуется с тем, что остальная часть проекта была командной работой, тогда как это направление ASRP вела от начала до конца:

- Спроектировала асимметричную **модель источника истины** (АТС авторитетна для живого состояния, база данных авторитетна для UI) и **трёхпутевую сходимоть**, которая поддерживает их согласованность.
- Построила полный поток **входа/выхода агента из очереди** с учитывающим huntgroup **автоселектом устройства** и двусторонним подтверждением.
- Построила **синхронизацию устройств** (состояние регистрации, назначение очереди, снимки истории) и **SIP-предоставление софтбокса** с зашифрованным хранением учётных данных.
- Решила задачу **корреляции звонков** через устойчивую цепочку SIP-Call-ID, включая **буфер повтора** на базе Redis для событий АТС не по порядку / дублирующихся.
- Построила слой **лечения на основе вебхуков** (включая обратное лечение и защиту активного звонка) и **плановый reaper устаревших логинов** (распределённая блокировка, единственное исполнение).
- Реализовала **OAuth2-клиент АТС** с конкурентно-безопасным обновлением токена и прозрачным повтором при 401.
- Сделала **взятие звонков устойчивым к сбоям** — с откатом локального взятия при неудачном переводе АТС.

- Поставила **реалтайм-обновления UI** через NATS / WebSocket для состояния агента и очереди.
- Возглавила инженерный цикл доведения до боевой устойчивости в середине 2026 года, который провёл интеграцию от рабочего прототипа через пилот у клиента.

14. Часто задаваемые вопросы (для разговоров с клиентами)

В: Какую базу данных вы использовали? MySQL через Drizzle ORM в TypeScript — PlanetScale serverless в продакшене и драйвер mysql2 против MySQL 9 локально, с разделением primary/replica и авторитетными чтениями состояния очереди, закреплёнными за primary.

В: Это то же самое, что ваша внутриплатформенная маршрутизация очередей? Нет. У платформы есть *своя собственная* встроенная маршрутизация. Этот проект интегрирует **внешнюю** облачную АТС, чтобы организации, уже использующие её для телефонии, могли управлять своими существующими очередями и агентами АТС из UI платформы. Это два независимых механизма, переключаемых для каждого номера телефона отдельно.

В: Две системы хранят состояние — как вы не даёте им разойтись? Асимметричная модель источника истины плюс три пути сходимости: платформа выполняет прямые мутации, АТС проталкивает вебхуки событий, которые почти в реальном времени лечат базу данных, а плановый reaper сверяется с авторитетным списком агентов АТС каждые 15 минут, чтобы уловить всё пропущенное. Самовосстановление работает в обоих направлениях.

В: Как вы сопоставляете событие звонка от АТС с нужным звонком платформы? Через цепочку устойчивого идентификатора: внешний SIP Call-ID — это единственный идентификатор, о котором договорились и медиашлюз, и АТС, поэтому платформа записывает его в запись звонка как индексированный ключ корреляции и сопоставляет с ним события АТС. События, приходящие до записи ключа, буферизуются в Redis и воспроизводятся, как только он становится доступен.

В: Что происходит, когда АТС пропускает отправку события? 15-минутный reaper независимо сверяет состояние с авторитетным списком агентов АТС, закрывая устаревшие логины и ресинхронизируя устройства. Вебхуки и reaper дополняют друг друга — ни один из них не является единой точкой отказа.

В: Может ли это случайно вывести кого-то из системы посреди звонка? Нет — сверка включает защиту активного звонка и общее определение «эффективного входа», используемое единообразно UI, вебхуками и reaper, так что временный или устаревший сигнал не может закрыть достоверный логин на звонке.

В: Как обрабатывается аутентификация к АТС? OAuth2 с токеном, кэшируемым в Redis, проактивным обновлением до истечения срока, дедупликацией внутри процесса плюс распределённой блокировкой Redis, чтобы несколько экземпляров никогда не обновляли токен одновременно, и прозрачным одноразовым повтором при истечении срока действия токена.

В: Какая проблема была самой сложной? Корреляция звонков — конкретно гонка, при которой быстро отвечающие звонки из очереди АТС генерируют своё событие звонка до того, как платформа записала ключ корреляции. Решена с помощью дизайна с буферизацией и повтором (§7.1).

В: Почему это больше не находится в активной разработке? Проект приостановлен, а не заброшен: после пилота у клиента экономика склонилась в пользу маршрутизации звонков этого клиента через собственный встроенный движок очередей платформы, и фронтенд-работа, построенная для этой интеграции, перешла напрямую. Сама возможность внешней АТС по-прежнему работает и остаётся доступной для организаций, уже использующих эту внешнюю АТС.

Анонимизированное описание проекта, подготовленное ASRP для использования в портфолио. Название клиента, проприетарный исходный код, внутренние идентификаторы, внутренние URL и операционно-эксплуатационные детали безопасности намеренно опущены. Более глубокие технические детали доступны по NDA.

1. **RingLogix / Netsapiens** — коммерческая white-label платформа облачной АТС / UCaaS (хостируемые очереди звонков, huntgroups, SIP-устройства и REST API), которую телеком-провайдеры перепродают собственным бизнес-клиентам. Упомянута здесь исключительно как цель сторонней интеграции; это публичный продукт. <https://www.netsapiens.com>↪
2. **RingLogix / Netsapiens** — коммерческая white-label платформа облачной АТС / UCaaS (хостируемые очереди звонков, huntgroups, SIP-устройства и REST API), которую телеком-провайдеры перепродают собственным бизнес-клиентам. Упомянута здесь исключительно как цель сторонней интеграции; это публичный продукт. <https://www.netsapiens.com>↪